

PCS956-TS-01-intro-and-overview

September 13, 2026

1 PCS956 - Time Series for ML - Foundations, EDA, and Modelling Pitfalls

In this module, we do **not** assume that you already know time series analysis. Many of you will have seen regression, classification, and standard ML methods, often under the assumption of independent and identically distributed data.

Time series break this assumption. Observations are ordered in time, often dependent on previous values, and affected by trends, seasonality, and regime changes. These features are central to robust modelling and evaluation.

Because we have limited time, we will **not** develop the full mathematical theory behind classical methods (for example ARIMA, spectral analysis, or state-space models). Instead, we will:

- focus on **conceptual understanding** of trend, seasonality, residuals, stationarity, and random-walk-like behaviour;
- cover enough structure and assumptions of classical models for you to **critically assess when and why they are valid, and where they can fail**;
- use these concepts to motivate **strong baselines** and **careful validation**;
- treat classical models more as **background knowledge** that informs our choice of baselines and transformations.

The aim is not for you to become time-series theorists, but to ensure that your research does not rely on naive temporal modelling or overstated claims of predictive skill. In particular, you should be able to:

- recognise when temporal structure matters;
- avoid common pitfalls when applying ML to time series;
- design sensible baselines and checks that match your research questions;
- argue clearly for the limitations of your conclusions in light of data quality, stationarity assumptions, and concept drift.

This lecture introduces core concepts for working with time series in applied machine learning: temporal indexing and finite samples, data quality and anomalies, visual exploratory data analysis (EDA), basic decomposition, stationarity, and common modelling pitfalls. The aim is to build a critical, scientifically grounded mindset before students begin modelling and forecasting in later lectures.

In this module, **not all time-series work is about forecasting**. Many important tasks involve classification (for example distinguishing earthquakes from underground nuclear tests using seismic

signals), anomaly detection (for example flagging unusual sensor behaviour), or structural modelling (for example summarising dependence and regime changes). The same core ideas—temporal indexing, data quality, EDA, decomposition, stationarity, and concept drift—apply across these different task types.

By the end of the lecture, students should be able to: - describe what a time series is and distinguish between univariate and multivariate series; - recognise common data-quality issues, anomalies, and rare events in temporal data; - carry out basic visual EDA for trend, seasonality, and autocorrelation; - understand simple decomposition into deterministic and stochastic components; - explain, at an intuitive level, stationarity and random-walk-like behaviour; - identify key modelling pitfalls and think critically about whether a series contains exploitable structure.

This sets the foundation for: - Lecture TS2: baseline methods, forecasting models, temporal validation, and evaluation; - Lecture TS3: anomaly detection, concept drift, multivariate dependence, and explainability.

1.1 1. Motivation and framing

Time series arise whenever we record measurements over time. They are central in many applied ML problems, yet they are often treated as if they were independent and identically distributed (i.i.d.) observations. This can lead to misleading conclusions.

Much of the ML and statistics you have seen so far typically assumes that:

- observations can be shuffled without changing their meaning;
- standard cross-validation and train/test splits are legitimate under random shuffling;
- performance metrics such as R^2 or accuracy summarise model quality without reference to time.

For time series, these assumptions break down. Shuffling destroys temporal structure, temporal dependence reduces the effective amount of **independent** information, and validation must respect time order. Even large datasets can be “thin” in terms of independent evidence once autocorrelation and non-stationarity are taken into account.

We will repeatedly return to rare and extreme events, because their absence from historical data is often the main reason why models fail in deployment: tail behaviour matters most for risk and decision-making, yet is typically least represented in the sample.

A key message of this lecture is that **almost everything you have learned so far assumes no time ordering**. We are now examining what breaks when we ignore that ordering, and what is required to make robust claims in temporal settings.

Naive thinking of the form “data plus model equals publication” is particularly dangerous in temporal settings. The temporal index introduces dependence, non-stationarity, and sensitivity to rare events and regime changes. These features strongly affect: - what can be learned from historical data; - how we should validate models; - how robust our conclusions are when the environment changes.

Throughout the lecture we emphasise: - careful EDA before modelling; - scepticism about apparent predictive skill on strongly autocorrelated data; - the importance of asking whether the series really contains exploitable structure.

Textbook examples often show clean, regularly sampled series with neat trends and periodic patterns. Real data are rarely so tidy:

- sampling may be irregular;
- sensors may fail or drift;
- units, scaling, and alignment across variables may be inconsistent;
- rare or extreme events may be missing entirely from the sample.

A central theme of this lecture is to **bridge the gap** between the neat conceptual picture and the messy reality, and to show how this affects modelling and evaluation.

1.1.1 1.1 Time series in students' domains

A sample from a time series can be almost any quantity recorded over time. Examples include:

- financial data: stock prices, exchange rates, bond yields;
- food prices and other economic indicators;
- population-related data: births, deaths, migration, education levels;
- epidemiological data: reported cases of a disease in a region;
- text and social media data: counts of tweets or specific phrases per day;
- meteorological data: temperature, precipitation, wind speed and direction;
- climatological data: long-term observations of temperature or rainfall;
- energy data: consumption, production, and prices for electricity, gas, coal, oil;
- medical and physiological data: heart rates, lab measurements, sensor readings;
- neuroimaging data: time series from fMRI or EEG;
- sound and video recordings.

Many student projects will involve combining several such series or focusing on a subset of variables. In some cases (assuming overlapping time indices and compatible units) different time series can be merged, or less relevant parts of a multivariate series can be removed, to focus on variables of interest. This practical step already raises questions about temporal alignment, sampling rates, and the structure of the resulting data.

1.1.2 1.2 Why worry about time-series-specific issues?

Suppose you have a dataset with a clear temporal index:

- you plot it and it looks vaguely like a stock price, or energy demand, or some sensor value;
- you recognise the shape from examples you have seen in blog posts, papers, or lectures.

A very natural workflow is:

1. Choose a tool you have seen used on similar-looking data (for example a popular ML model or a standard forecasting method).
2. Fit the model to your series.
3. Compute familiar indicators (for example R^2 , mean squared error, classification accuracy).
4. The numbers look good, the plots look convincing.
5. You move on: deploy the model to production, or write up a publication, or treat the result as a solid scientific finding.

This workflow feels attractive:

- it reuses methods you already know;

- it gives clear numbers and visualisations;
- it avoids “complicating matters” with time-series-specific concerns.

The problem is that **time series can make superficial success very easy**:

- strong autocorrelation and smoothness can inflate performance metrics;
- naive validation procedures can leak information from the future into the training set;
- apparent improvements over simple baselines may vanish under proper temporal evaluation.

It is entirely possible to build a model that looks excellent on paper yet has learned almost nothing about the underlying temporal structure. This risk is not limited to beginners; it can hit anyone who:

- treats time-ordered data as if it were i.i.d.;
- evaluates on levels only, without careful baselines;
- ignores transformations that reveal underlying random-walk-like behaviour.

The rest of this module is about understanding *why* this happens, *how* to detect it, and *what* to do instead.

1.2 2. What is a time series?

A time series is a collection of time-indexed observations X_t . The index might be discrete or continuous, but in most applied settings we work with discrete time and a roughly constant sampling interval:

$$\dots, X_{-2}, X_{-1}, X_0, X_1, X_2, \dots$$

The observations can be univariate or multivariate. In the multivariate case we write

$$X_t = (X_{1t}, \dots, X_{dt}),$$

where d is the number of variables recorded at each time point. When d is large, direct modelling of all components can be difficult in practice. Dimension reduction or selection may be necessary, and methods must respect the temporal nature of the data.

In real applications, time series appear in many forms: - regularly sampled sensor readings; - irregularly spaced events or measurements; - single series or collections of related series; - raw signals or transformed features.

Being explicit about the temporal index and the data structure is the first step towards sensible modelling.

1.2.1 2.1 Temporal indexing and finite samples

In practice we only observe a finite sample from a time series:

$$X_1, X_2, \dots, X_{N-1}, X_N.$$

Even if the underlying process could in principle run for ever, we only see this one finite realisation. This has consequences:

- estimates and models are based on limited information;
- rare or extreme events may not appear in the sample;
- uncertainty statements are constrained by what we have actually observed.

Given only one observed sample from a real phenomenon, resampling techniques such as block bootstrapping may sometimes be used to obtain confidence intervals for estimates or forecasts.

Samples can also be simulated from a parametric model to sanity-check algorithms: if we generate data from a known model and apply our method, do we recover the expected behaviour?

Low-quality samples can lead to disastrous conclusions. The reliability of results depends heavily on sample size, missing observations, and the presence or absence of extremes. Later sections will revisit these issues under data quality, anomalies, and rare events.

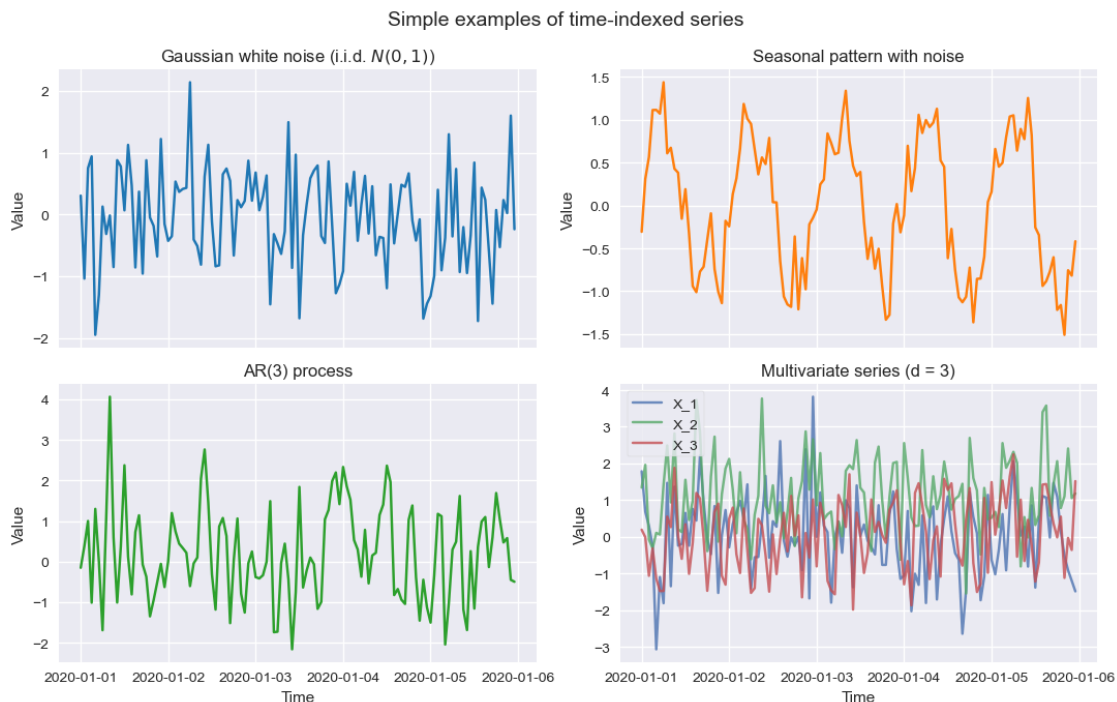
1.2.2 2.2 Univariate and multivariate structure

A univariate time series records a single quantity over time. A multivariate series records several quantities at each time point, often with complex dependencies between components.

Examples: - univariate: daily closing price of a single stock; - multivariate: vector of prices for several stocks, or a set of environmental variables (temperature, humidity, wind speed) recorded at the same times.

When d is modest, modelling all variables jointly may be feasible and desirable. When d is large, direct modelling of all components can be difficult: - many potential relationships to explore; - higher risk of spurious discoveries; - computational and interpretability challenges.

Dimension reduction or variable selection can help, but any such method must respect time order and dependence. Later lectures (especially Lecture TS3) will return to multivariate and nonlinear dependence, and to techniques that operate in the time or frequency domain to reduce complexity.



1.3 3. Data quality, anomalies, and rare events

Real-world time series are often messy. Before any modelling, it is essential to assess: - how the data were collected and recorded; - whether sampling is regular or irregular; - the presence of missing values, anomalies, and regime changes; - whether important rare or extreme events are represented.

These issues strongly affect the reliability of forecasts, classification results, and scientific conclusions.

1.3.1 3.1 Temporal data structures

Temporal data come with structures that are easy to ignore but crucial for analysis:

- regular vs irregular sampling:
 - regular sampling: observations at roughly constant intervals (for example every minute, hour, day);
 - irregular sampling: event-driven measurements or data with gaps and bursts;
- alignment across variables:
 - merging multiple series requires compatible time indices and careful handling of missing times;
 - different sensors may record at different frequencies or with different delays.

Resampling and alignment operations (for example aggregating to daily totals or interpolating to a common grid) can simplify analysis, but they also introduce assumptions and can hide regime changes. Sudden shifts due to interventions, failures, or policy changes may look like anomalies, but they often represent genuine changes in the underlying dynamics.

1.3.2 3.2 Missing data, anomalies, and nonsense values

The quality of a sample has a strong impact on any conclusions drawn from it:

- missing observations: some X_t may not have been recorded properly;
- sensor errors and nonsense values: physically impossible readings, jumps or spikes;
- regime changes: sudden shifts due to interventions, failures, or policy changes;
- rare or extreme events: important phenomena (market crashes, extreme storms, rare failures) may not appear in the sample at all.

Missing values can sometimes be handled by interpolation or imputation, but these steps introduce additional assumptions. Rare/extreme events are particularly problematic: a forecasting or decision algorithm that has never seen such events may perform badly when they occur in reality. An apparently highly profitable trading strategy can look brilliant in backtests yet fail catastrophically when confronted with a regime change or extreme event that was absent from the training data.

When dealing with high-dimensional or noisy data it is easy to find spurious patterns. Examples include:

- the multiple comparisons problem: if we test many hypotheses, some will appear significant by chance alone;

- widely cited cautionary examples such as the “dead salmon” fMRI study, which highlight how naive analysis can produce apparently meaningful results from nonsense data.

These examples are reminders to:

- be clear about how many models or hypotheses have been tried;
- use appropriate corrections or sceptical interpretation;
- avoid overclaiming on the basis of a single exploratory pattern.

In high-dimensional, noisy temporal data (for example fMRI, industrial sensors, environmental monitoring), these issues are amplified:

- long series and many variables make it easy to “go fishing” for patterns;
- flexible ML models can fit subtle-looking structure even when the underlying signal is weak or meaningless;
- regime changes and sensor artefacts can masquerade as genuine discoveries.

The more data and models we have, the easier it becomes to fool ourselves. Time series make this worse because dependencies are subtle and regime changes are real. Careful documentation of data quality and modelling choices is therefore essential.

1.3.3 3.3 Rare and extreme events

Rare and extreme events are often the most important for decision-making, yet they may be absent from the observed sample. Examples: - financial market crashes; - extreme storms or floods; - rare equipment failures or medical crises.

Their absence limits what we can conclude from historical data:

- we may underestimate risk or variability;
- models may be overconfident in regimes they have seen;
- extrapolation beyond the observed range becomes highly uncertain.

A forecasting or control algorithm that has never seen such events may perform badly when they occur. This is a central challenge for time series in safety-critical and high-stakes domains.

For decision-making, missing extremes can be more dangerous than noisy everyday variability:

- a flood-risk model built on a few decades of river data may never have seen the most extreme events that matter for infrastructure design;
- a trading strategy backtested over a calm market regime can crumble during a crisis;
- a medical monitoring system trained on routine hospital data may underperform in rare emergency situations.

Time series models trained on a quiet history may be precisely wrong when we need them most. Being aware of what *is not* in the historical record is as important as analysing what *is*.



1.4 4. Visual EDA for time series

Time series appear in many different use cases, which motivate different tasks:

- classification:
 - seismic data: earthquake vs nuclear test (for example the benchmark dataset available via the R package `astsa`);
 - speech recognition: which language, what was said;
 - medical imaging and signals: cancer vs benign, normal vs abnormal heartbeat;
- optimisation and control:
 - planning a robot trajectory based on recorded human movement;
- modelling underlying structure:
 - building models that summarise trend, seasonality, and dependence;
- forecasting:
 - tomorrow's weather, next week's demand;
 - long-term climate indicators (average temperature or storm counts decades ahead);
 - financial quantities such as stock prices or interest rates;
- nowcasting:
 - real-time decisions based on delayed or partial measurements;
- causal questions:
 - understanding which changes in one series help explain or cause changes in another.

Different tasks emphasise different aspects of the data and require different validation schemes and baselines. Visual EDA is a common starting point across tasks. Before fitting models, we should:

- plot the raw series;
- look for trend, seasonality, and anomalies;
- consider transformations (for

example differences, logs); - examine autocorrelation at an intuitive level.

1.4.1 4.1 Plotting raw series

Simple time plots are often the most informative first step: - plot levels X_t against time to see overall patterns; - optionally plot differences (for example $X_t - X_{t-1}$) to highlight changes and reduce strong trends.

Visual inspection can reveal: - trends and cycles; - obvious anomalies and missing data; - potential regime shifts or changes in variability.

These plots are not formal tests, but they guide which models and diagnostics are worth trying.

1.4.2 4.2 Visual checks for trend, seasonality, and anomalies

When examining time plots and related graphics, it is useful to ask:

- Trend:
 - is there a long-term upward or downward movement?
 - is the mean roughly stable or clearly drifting over time?
- Seasonality:
 - are there regular patterns (daily, weekly, yearly) that repeat?
 - do certain times of day, days of the week, or months of the year show consistent differences?
- Anomalies and level shifts:
 - are there sudden jumps, drops, or spikes;
 - do we see persistent level shifts that might correspond to regime changes;
 - are there obvious sensor failures or nonsense values?

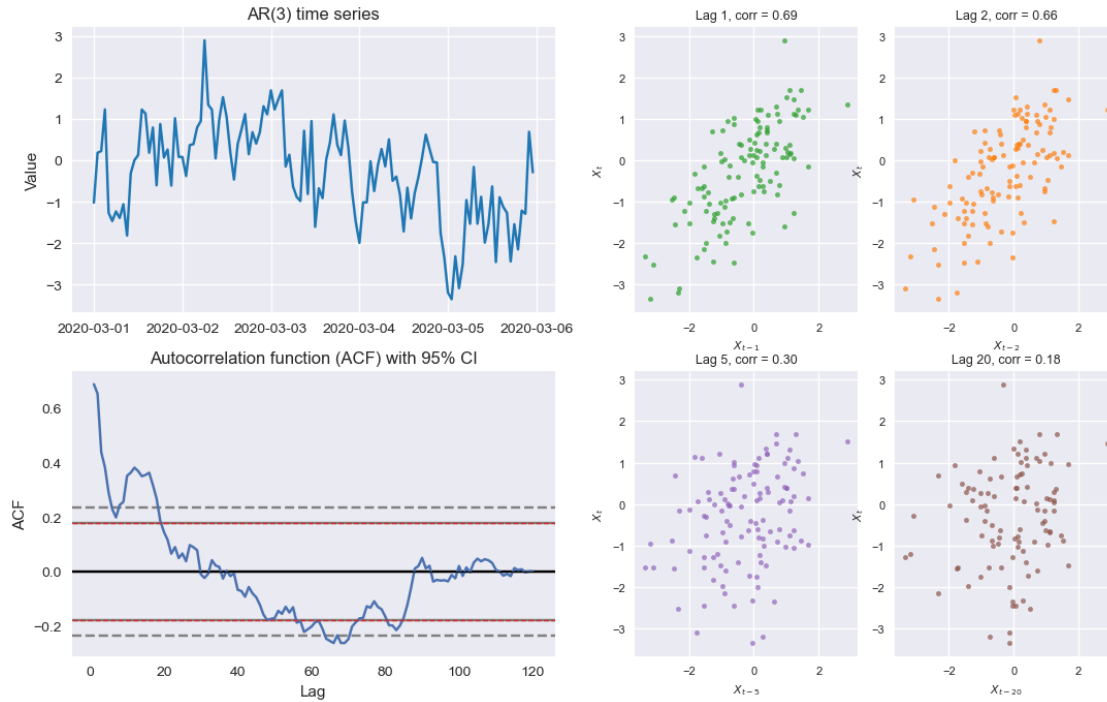
These visual checks help decide whether simple baselines (for example persistence) are plausible, whether more flexible models are needed, and whether data cleaning or segmentation is required.

1.4.3 4.3 Autocorrelation at an intuitive level

Autocorrelation measures how strongly current values relate to past values. Informally: - if high values tend to follow high values, and low values tend to follow low values, the series is positively autocorrelated; - if high values tend to follow low values and vice versa, negative autocorrelation may be present; - if no clear pattern exists, autocorrelation may be weak.

Autocorrelation plots (for example autocorrelation functions, ACF) show correlation at different lags. At this stage we focus on the intuition: - strong autocorrelation implies persistence and potential predictability; - weak autocorrelation makes forecasting harder and persistence baselines less effective; - complex patterns may signal seasonality or other structures.

Later lectures will formalise these ideas and introduce more sophisticated tools.



1.4.4 4.4 Train, validation, and test splits in time series

In machine learning workflows it is common to split data into:

- **Training set**
Used to fit model parameters (for example regression coefficients, tree splits, or ARIMA parameters).
- **Validation set**
Used to tune model choices (for example hyperparameters, lag order, regularisation strength) and compare different models.
The validation set is *not* used to fit the final parameters; it serves as a held-out dataset for model selection.
- **Test set**
Used only at the end to assess the performance of the chosen model on data that were not used for training or validation.
This set should remain untouched while models and hyperparameters are chosen, to avoid information leaking back into training.

In time series, these splits must respect **time order**. A typical pattern is:

- training: earliest part of the series;
- validation: a later window in the same regime;
- test: an even later window, still in the same regime.

All three can be taken from one regime of the series. If the underlying dynamics later change (regime shift or concept drift), the test performance in the old regime can still be good, while

future performance on the new regime may be poor.

A critical issue in time-series ML is **leakage**: information from the future leaking into the training or validation procedure. Examples include:

- shuffling the data randomly before splitting, so that training points come from later times than validation or test points;
- using the entire series (including test and future points) to compute features or scalings that are then applied to the training set (for example normalising by a global mean and variance);
- repeatedly peeking at the test set while choosing models or hyperparameters, effectively turning it into another validation set.

Leakage can also occur more subtly in series with **long-term memory** or strong autocorrelation. If training ends at time t and validation or test starts immediately at $t + 1$, the model may implicitly “remember” information from the training window far into the validation/test window. In such cases, practitioners sometimes insert a **quarantine gap** between train, validation, and test windows: for example,

$$\text{Train: } [t_0, \dots, t_1], \quad \text{Gap: } [t_1 + 1, \dots, t_1 + g], \quad \text{Validation/Test: } [t_1 + g + 1, \dots, t_2].$$

The gap reduces the influence of long-range dependence from the training period on the evaluation period, at the cost of discarding some data.

Leakage can make a model appear much better than it truly is, because it has indirectly been given access to the “future” it is supposed to predict. In temporal settings, splits must respect time order, test data should remain untouched until final evaluation, and quarantine gaps may be considered when long-term memory is a concern.

1.5 5. Decomposition: trend, seasonality, residual

It is often helpful to view a time series X_t as the sum of a deterministic component and a stochastic (random) component:

$$X_t = \text{deterministic}(t) + \text{stochastic}(t).$$

The deterministic part is a function of time. It is common to split it into: - a trend function capturing long-term movement; - a periodic or seasonal function capturing regular cycles (for example daily, weekly, yearly).

The stochastic part contains the remaining variability after accounting for trend and seasonality. It is often treated as “noise”, but this noise can still have important structure: - autocorrelation (dependence across time lags); - non-linear temporal and spatial dependencies; - changing variance over time.

In practice we estimate these components from the observed sample, for example by: - first fitting a simple deterministic model (trend plus seasonality); - then using residuals as the basis for investigating the stochastic part.

1.5.1 5.1 Trend and seasonality

Trend refers to long-term movement in the mean level of the series: - persistent upward or downward trajectories; - slow changes over years or decades.

Seasonality refers to regular, repeating patterns: - daily cycles (for example electricity demand); - weekly cycles (for example retail sales); - yearly cycles (for example temperature or rainfall).

Many practical models start by capturing trend and seasonality, either explicitly (for example regression on time and seasonal indicators) or implicitly (for example through smoothing or decomposition). Identifying these components visually is an important part of EDA.

1.5.2 5.2 Residuals and structure vs noise

After removing estimated trend and seasonality, the remaining series (residuals) should ideally look like noise with no obvious structure: - roughly constant mean (around zero); - roughly constant variance; - limited autocorrelation.

When residuals still show clear patterns, this suggests remaining structure: - strong autocorrelation may call for specific time-series models (for example ARMA-type or state-space models); - changing variance may suggest heteroscedasticity or volatility models.

A chaotic sequence is not the same as a random sequence:

- A chaotic sequence is generated deterministically from a dynamical system of the form $x_{n+1} = f(x_n)$, where f is a smooth function on $\mathbb{R}^d$.
- Chaos is typically characterised by sensitive dependence on initial conditions: arbitrarily close initial values x_{01} and x_{02} can produce trajectories that diverge rapidly.
- If we start from the same initial value x_0 twice, we obtain exactly the same sequence; the apparent randomness comes from sensitivity, not from genuine stochasticity.

From a practitioner's perspective, chaotic dynamics can be **effectively unpredictable** beyond short horizons, even though they are deterministic in principle. This matters conceptually, but it does not guarantee better long-horizon predictability than genuinely random noise.

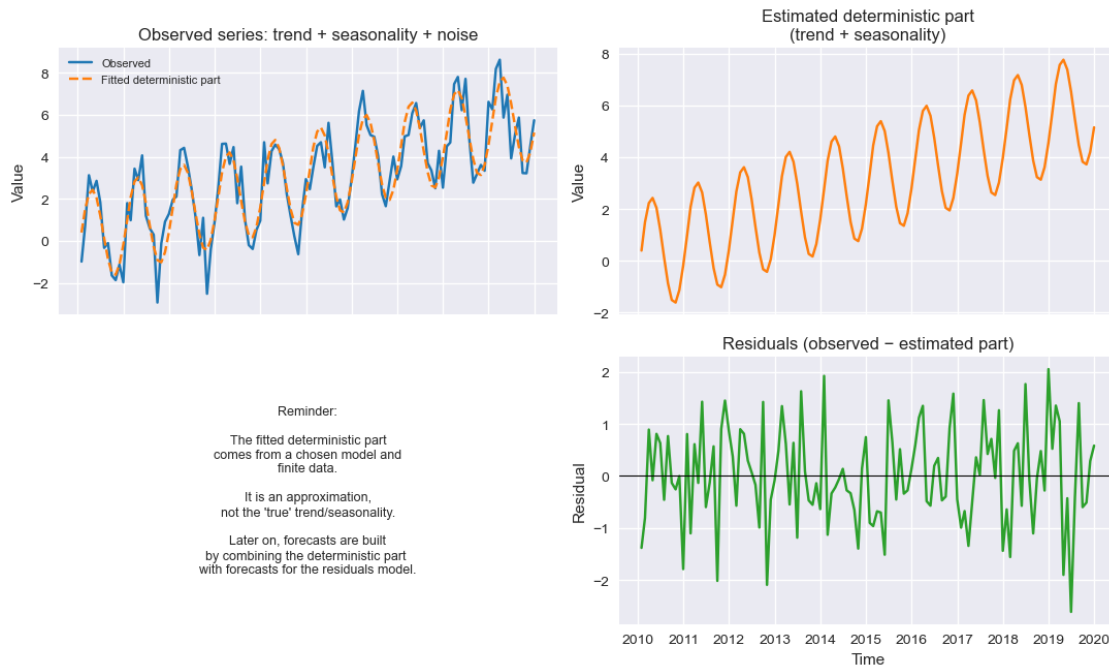
Looking irregular does not prove randomness: deterministic chaotic systems can produce series that look random. Looking structured does not prove predictability: random walks can show apparent trends and patterns in levels that vanish after simple transformations. Visual impressions alone are therefore not enough. We need:

- decomposition into trend, seasonality, and residuals;
- appropriate transformations (for example differences, logs);
- and strong baselines,

to judge whether a series contains genuinely exploitable structure or is close to noise or random-walk behaviour. Residual analysis is one way to probe where a series sits on this spectrum.

Section 6.3 gives a concrete example with random walks and “stock-like” series, illustrating how visually rich behaviour in levels can be misleading and why transformations and baselines are essential for honest assessment.

Simple trend–seasonality decomposition on a toy series



1.5.3 5.3 Connecting back to the original series and scale

In practice, our main interest is often in the original series (for example prices, demand, river flows), not in the transformed or residual series we use for modelling. This creates an important step that is easy to overlook:

- we may **fit models to residuals** after removing trend and seasonality;
- we may **fit models to transformed data**, such as differences or logarithms;
- but forecasts and decisions usually need to be expressed on the **original scale and in the original context**.

For example:

- if we model differences $\Delta X_t = X_t - X_{t-1}$, a forecast of ΔX_{t+1} must be combined with X_t to obtain a forecast of X_{t+1} ;
- if we model $\log X_t$ (for strictly positive data), we need to exponentiate forecasts and account for the effects of the log transformation when interpreting errors or confidence intervals;
- if we model residuals after removing trend and seasonality, we must add back the estimated trend and seasonal components to obtain a forecast of the original series.

It is therefore important to **keep track of the full chain of transformations**:

original series $\xrightarrow{\text{transform}}$ detrended / deseasonalised / transformed series $\xrightarrow{\text{model}}$ predictions on transformed scale

Horror stories arise when:

- many transformations are applied without a clear record (for example multiple filters, differences, logs, normalisations);
- models are evaluated on the transformed scale, but results are reported on the original scale without proper back-transformation or adjustment;
- the connection between the prediction target (what we actually care about) and the modelled quantity (residuals or transformed values) is unclear.

A practical rule is:

- **define the target on the original scale first** (for example “next-day demand in MWh”);
- **be explicit about each transformation** applied before modelling;
- **implement and check the inverse transformation** so that predictions can be meaningfully mapped back to the original situation.

Later code templates (Section 10) will illustrate how to:

- apply transformations and decomposition,
- fit models on residuals or transformed series,
- and correctly reconstruct forecasts and diagnostic plots on the original scale.

1.6 6. Stationarity and random-walk intuition

Stationarity is a central concept in time series analysis. Intuitively, a stationary series has statistical properties (mean, variance, autocorrelation) that do not change over time. Many classical models assume some form of stationarity.

Random-walk-like behaviour, in contrast, involves strong persistence and non-stationarity. Such series can be very hard to predict beyond simple baselines, and they challenge naive applications of ML methods.

1.6.1 6.1 Stationary vs non-stationary series

A time series is (weakly) stationary if: - its mean is constant over time; - its variance is constant over time; - its autocovariance $\gamma(h)$ depends only on the lag h , not on the absolute time index t .

Non-stationary series violate one or more of these conditions. Common examples include: - series with trends or changing levels; - series with changing variance (for example volatility clustering in finance); - series with evolving dependence structures.

In practice, almost every real series you encounter will be non-stationary in raw levels, because of: - trends driven by technology, policy, or climate change; - seasonal effects (daily, weekly, yearly); - interventions, regime changes, and structural breaks.

Stationarity is therefore best viewed as a **modelling stance**, not a property magically given by the data. We often try to *approximate* stationarity by:

- removing trend (for example via regression on time or smoothing);
- removing seasonality (for example via seasonal indicators or decomposition);
- differencing (for example working with $\Delta X_t = X_t - X_{t-1}$ rather than X_t).

Random-walk-like series are classic examples of non-stationarity in levels: they typically require differencing before stationary models such as ARMA can sensibly be applied. ARIMA models

encode this idea explicitly by combining autoregressive (AR) terms, moving-average (MA) terms, and integrated (I) differencing:

- autoregressive (AR) terms model dependence of X_t on its own past values (for example X_{t-1}, X_{t-2});
- moving-average (MA) terms model dependence on past shocks or residuals (for example $\varepsilon_{t-1}, \varepsilon_{t-2}$);
- the integrated (I) part corresponds to differencing to handle non-stationarity in levels.

Information criteria such as AIC and BIC are useful tools for comparing ARIMA models, but they are not magic oracles:

- they approximate a trade-off between goodness of fit and model complexity;
- they depend on the finite sample we happen to observe;
- they can favour a slightly wrong order if the sample is short or noisy.

Even when the true data-generating process is an AR(1), a finite sample can make an ARIMA(2,0,0) or ARIMA(1,0,1) look slightly better by AIC. In practice this is acceptable as long as the selected model captures the main structure and behaves reasonably under validation. The key is to remain aware that model orders are **chosen**, not revealed, and that concept drift and regime changes can invalidate them over time.

In practice, tests such as the Augmented Dickey–Fuller (ADF) test are often used as heuristics for stationarity or unit-root behaviour. These tests rely on idealised assumptions (for example linear dynamics and stable error distributions) and can give misleading comfort in the presence of structural breaks, strong non-linearity, or drift. They should be treated as **guides**, not oracles.

In this module, we mainly use stationarity as:

- an idealised reference point for thinking about dependence and variance;
- a motivation for transformations and decomposition;
- and a reminder that we must ask whether “rough stationarity after simple transformations” is plausible in our domain, or whether deeper non-stationarity and drift are unavoidable.

1.6.2 6.2 Random-walk-like behaviour and persistence

A simple random walk can be written as:

$$X_t = X_{t-1} + \varepsilon_t,$$

where ε_t is a noise term (for example independent with mean zero). Such series: - exhibit strong persistence: current values are close to recent past values; - are typically non-stationary in levels; - may be stationary in differences (for example $\Delta X_t = X_t - X_{t-1}$).

Random walks are classic examples of martingales: conditional on present information, the expected next value equals the current value,

$$\mathbb{E}[X_{t+1} \mid X_t, X_{t-1}, \dots] = X_t.$$

This is one reason why persistence forecasts are often hard to beat in random-walk-like domains: in such settings, “tomorrow will be like today” is, in a precise sense, the best unbiased forecast

available under the model.

In many applications (for example financial prices), random-walk-like behaviour means: - simple persistence forecasts (for example “tomorrow will be like today”) can be hard to beat; - apparent forecasting skill on levels may reflect persistence rather than genuine structure; - transformations (for example differences, returns) are often needed before modelling.

Understanding random walks and persistence is crucial for interpreting performance metrics and for avoiding illusions of skill on strongly autocorrelated data.

1.6.3 6.3 A cautionary example: a “stock-like” series generated by a random walk

Consider a series that visually resembles a stock price:

- it fluctuates up and down over time;
- it shows periods of apparent trends and reversals;
- it looks like the kind of data many forecasting models are applied to.

We can generate such a series artificially by taking a very simple random walk:

$$X_t = X_{t-1} + \varepsilon_t, \quad \varepsilon_t \in \{-1, +1\},$$

where each increment ε_t is either +1 or -1 at each step. This is a **purely random process** in differences:

$$\Delta X_t = X_t - X_{t-1} = \varepsilon_t.$$

There is no hidden pattern in ε_t beyond randomness; the process has no exploitable structure in the increments.

Visually, the series X_t may look rich:

- rises and falls suggest “bull” and “bear” periods;
- short apparent trends may tempt us to believe in predictability;
- the overall path looks similar to many real financial or sensor series.

If we:

- fit a complex ML model to X_t as levels,
- evaluate using familiar metrics,
- do not compare to a persistence or random-walk baseline,

we might easily convince ourselves that the model has “found structure”.

But once we difference the series and look at ΔX_t :

$$\Delta X_t = X_t - X_{t-1} = \varepsilon_t,$$

it becomes clear that:

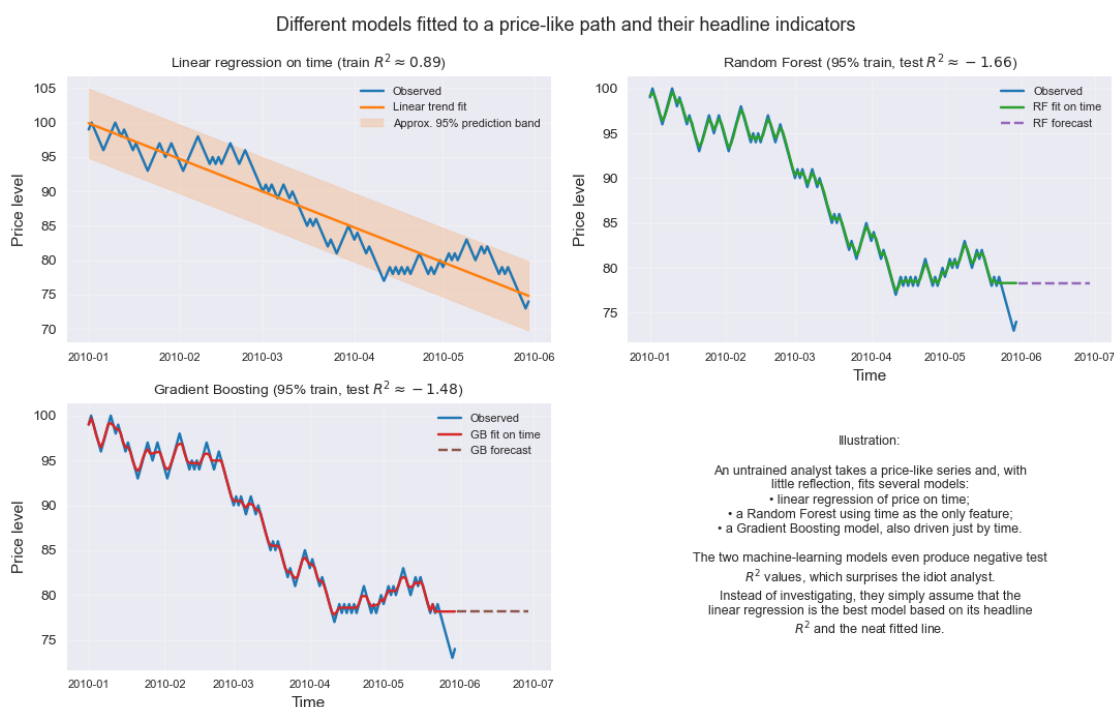
- each increment is just +1 or -1 with no temporal pattern;

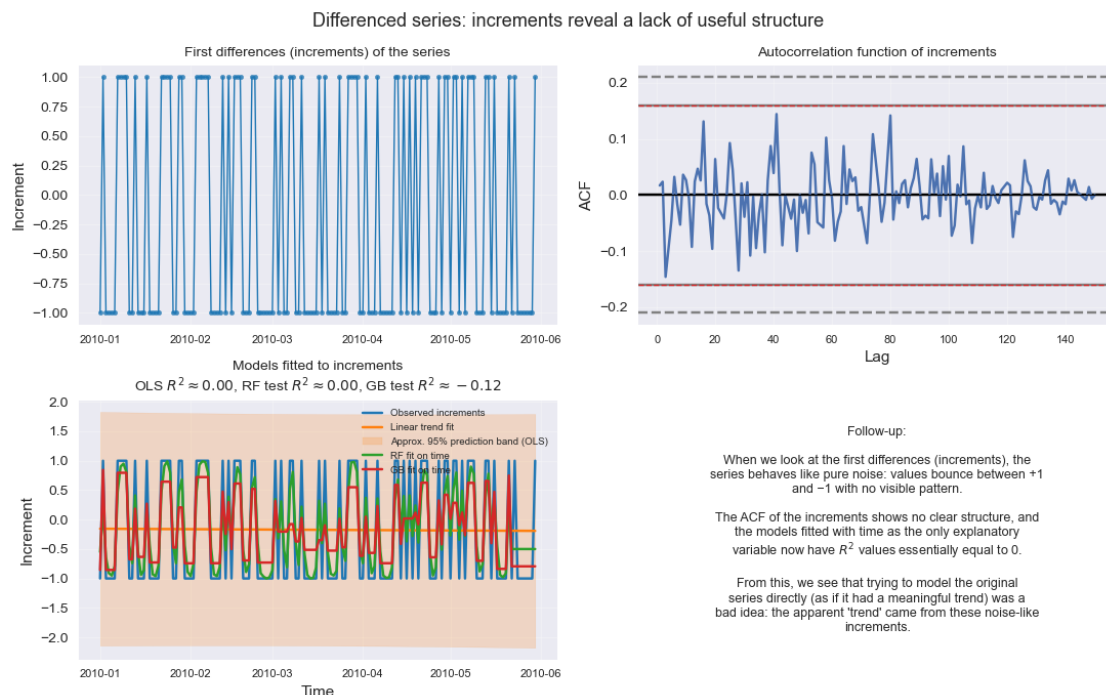
- any apparent forecasting skill on levels is largely an **illusion created by persistence and smoothness**;
- simple baselines that respect the random-walk nature (for example persistence or modelling differences) can reveal that the model has not learned anything fundamentally useful.

This example illustrates why:

- visual richness in levels does not guarantee exploitable structure;
- evaluating models on raw levels alone is often misleading;
- transformations (differences, returns, residuals) and **strong baselines** are essential for honest assessment.

This connects directly to the decomposition and residual analysis ideas in Section 5: once we look at differences (increments) instead of levels, the apparent trend largely disappears and the lack of exploitable structure becomes visible.





1.7 7. Modelling pitfalls and scientific mindset

Time series ML can appear attractive: large datasets, sophisticated models, and impressive visualisations. However, temporal dependence, non-stationarity, and data-quality issues make it easy to overstate predictive skill.

This section emphasises: - the limitations of applying off-the-shelf ML methods to temporal data; - illusions of skill on autocorrelated series; - the importance of asking whether there is exploitable structure beyond simple baselines.

Overall, we must earn our confidence in a time-series model by demonstrating that it beats appropriate baselines on the right transformations, under temporal validation, and on realistic residual diagnostics.

1.7.1 7.1 Naive use of ML models on temporal data

Time series are complicated to work with because dependence structures over time violate the independence assumptions underlying many standard ML algorithms. If a time series has no temporal dependence, meaning each observation X_t is independent of the others, we are back to the familiar situation of independent samples from a distribution.

However, many algorithms in statistics and machine learning are designed for independent observations. Applied naively to time series, such methods can fail badly:

- estimates of uncertainty can be too optimistic;
- standard cross-validation that shuffles data over time can leak future information into training;

- models may appear to perform well on levels but simply exploit persistence rather than genuine predictive structure.

In some cases, methods can be adapted to time series by: - transforming the time series problem into a supervised learning problem (for example using lagged features and sliding windows); - using evaluation schemes that respect time order (for example walk-forward validation).

Relevant tools include decision trees, Random Forests, and gradient boosting methods. These can be applied to time series once appropriate transformations and validation strategies are in place. For example, discussions of using gradient boosting methods such as XGBoost for time series forecasting emphasise: - the need to construct supervised-learning features from time series data; - the use of walk-forward or rolling-origin validation instead of random k -fold cross-validation.

1.7.2 7.2 Illusions of skill on autocorrelated data

Strong autocorrelation and persistence can make naive models look good:

- a persistence baseline $X_{t+1} = X_t$ may already achieve low error on smooth series;
- complex ML models trained on levels may appear to outperform simple baselines while in fact exploiting the same persistence.

Typical mistakes:

- evaluating models on levels without comparing to simple baselines;
- failing to check performance on transformed series (for example, differences) where persistence is less dominant;
- ignoring temporal validation, leading to leakage and overoptimistic metrics.

The key question is not “how high is the R^2 ?” but: *> Does the model capture structure beyond persistence and trend, and does it generalise under realistic temporal shifts?*

A negative R^2 indicates that the model performs **worse than a trivial baseline** that predicts the sample mean of the training data. On a series with little exploitable structure:

- simply predicting the mean level can outperform the fitted model;
- in many time-series settings, even simpler baselines such as **last observation carried forward** (“tomorrow today”) may be better.

More generally, whatever metric you use, always compare to baselines:

- if you use R^2 , then $R^2 \leq 0$ means that predicting the mean is as good as or better than your model;
- if you use error metrics such as RMSE or MAE, compare them to simple baselines (mean prediction, last-observation-carried-forward). If the baseline has **lower** RMSE/MAE than your model, the baseline is better.

If a simple baseline beats your model, that is a strong warning sign:

- the model is not extracting useful structure beyond the baseline;
- you may be better off using the baseline itself;
- or reconsidering whether the series contains any exploitable structure at all.

Prediction intervals for different models In the random-walk example, the linear regression on time comes with an approximate 95% prediction band, while the Random Forest and Gradient Boosting models are shown with point predictions only.

This is deliberate:

- For **ordinary least-squares (OLS)** regression, packages such as `statsmodels` have closed-form expressions for both confidence intervals (for the mean) and prediction intervals (for new observations). Once the model is fitted, we can obtain prediction intervals directly via `get_prediction(...).summary_frame(...)`.
- For **machine-learning models** such as Random Forests and Gradient Boosting in their standard scikit-learn form, the API provides only point predictions $\hat{y}$. These models do not, by default, estimate the full conditional distribution of $Y \mid X$, and there is no built-in analytic formula for prediction intervals.

It is possible to construct prediction intervals for ML models, but this requires additional methods, for example:

- quantile regression variants (e.g. quantile Gradient Boosting);
- conformal prediction or dedicated interval-estimation wrappers;
- bootstrap or ensemble-based approaches that approximate uncertainty.

These methods add conceptual and technical complexity. In this lecture we focus on:

- using OLS prediction intervals to illustrate uncertainty around a simple trend line;
- using RF and GB primarily to discuss overfitting, baselines, and negative R^2 on autocorrelated data.

In later work, if you want prediction intervals for ML models, you should be explicit about which method you use (quantile, conformal, bootstrap, etc.) and how its assumptions and guarantees compare to the simpler OLS case.

1.7.3 7.3 Asking whether there is exploitable structure

Models are important tools for classification and forecasting, but: - “All models are wrong, but some are useful” (George Box) — and in time series, every model is conditional on the historical window used to fit it; - verifying that assumptions are adequately satisfied on a given sample is often difficult; - nonparametric and machine learning methods can help, but they too rely on data quality and appropriate validation.

The quality of any result depends on the training data. Problems occur when: - the dataset is too small; - important patterns, especially rare events, are not represented; - the underlying dynamics change over time.

Usefulness in temporal settings depends on how the future differs from the past window used for training. A model can be locally accurate within one regime yet misleading or dangerous when the underlying dynamics change. This is why baselines, drift awareness, and model maintenance are central themes in this module.

Concept drift refers to situations where the statistical properties of the target variable or its relationship to inputs change over time. A forecasting model that performs well on historical data may degrade as the environment evolves. Examples include:

- industrial monitoring: sensors are replaced or recalibrated, operating procedures change, new equipment is introduced;
- energy systems: market rules or demand patterns change, new technologies are adopted;
- environmental and socio-economic data: long-term trends and policy interventions alter the behaviour of the series.

In such cases:

- continuous monitoring of performance is essential;
- refitting or retraining on updated data may be required;
- rigid “train once and deploy for ever” strategies are rarely adequate.

A scientific mindset asks: - what structures are present in the series, and are they stable over time; - whether simple baselines already explain most apparent performance; - how sensitive conclusions are to data-quality issues and model choices; - how likely concept drift is in the domain, and what maintenance strategy is appropriate.

Lecture TS3 will return to concept drift and model maintenance in more detail, including:

- tools for detecting drift and regime changes;
- strategies for monitoring and retraining;
- and the relationship between drift, anomalies, and multivariate structure.

1.7.4 7.4 ARIMA order selection via AIC in a toy drift example

In the code below, the old regime is generated from a known AR(1) process. However, we deliberately do not force the model to be AR(1). Instead, we:

- scan a small grid of ARIMA(p,d,q) models;
- choose the order with lowest AIC on the train+validation window.

For one particular finite sample (here seed = 94), this procedure selects an ARIMA(2,0,2) model, even though the true data-generating process in the old regime is AR(1). This illustrates the point made in Section 6.1:

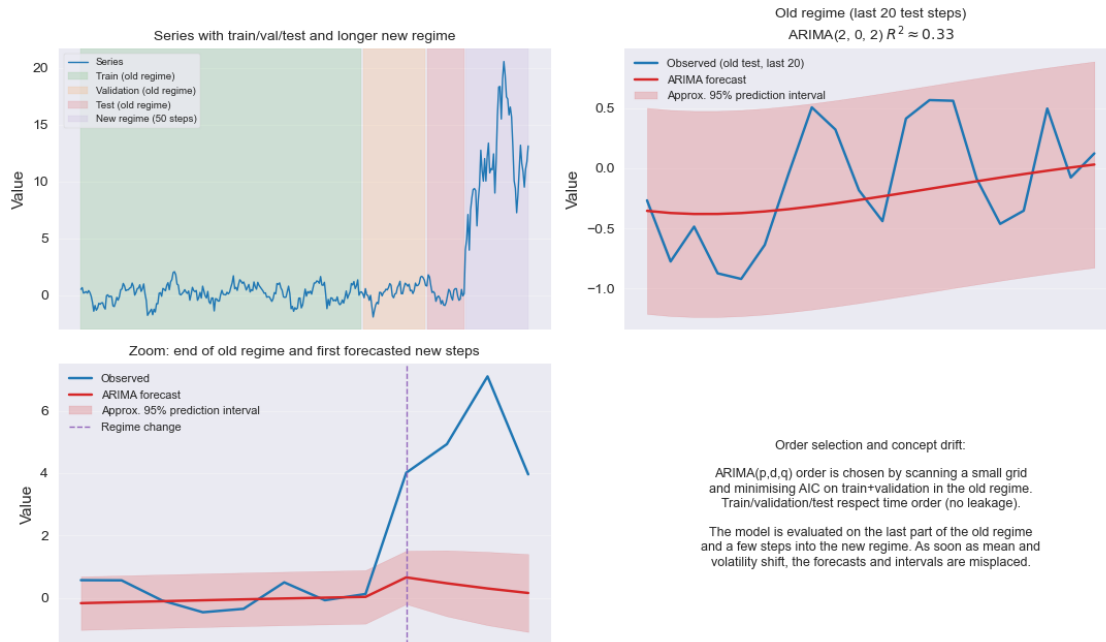
- **model orders are chosen from data, not revealed by an oracle;**
- information criteria such as AIC can favour a slightly misspecified order;
- what matters is whether the selected model behaves reasonably under validation, not whether it coincides with the underlying theoretical process.

We then show that this “best AIC” model, which fits the old regime reasonably well (R^2 around 0.33), fails badly as soon as the regime changes.

The train/validation/test setup in this example follows the principles in Section 4.4: splits respect time order and all three windows lie in the old regime.

```
Selected ARIMA order by AIC: (2, 0, 2)
Best AIC: 332.62889496903904
Old regime test R^2: 0.329
New regime R^2 (first 4 steps): -13.177
```

ARIMA order selection via AIC and concept drift (longer new regime, limited forecast horizon)



1.8 8. Time-series mini-project: aims and starting points

The time-series mini-project focuses on **one dataset** and **one main question**, using tools introduced in this lecture and expanded later. The emphasis is on a **critical, scientifically sound framework**, not on guaranteeing impressive predictive performance, and the assessment is on a pass/fail basis.

The main question does **not** have to be forecasting. Acceptable project types include, for example:

- **forecasting** tasks (e.g. short-term demand, climate indices, financial quantities);
- **classification** tasks (e.g. earthquake vs nuclear test from seismic data, normal vs abnormal behaviour in medical or industrial signals);
- other time-series tasks where temporal structure matters (e.g. change-point detection, anomaly detection, or simple modelling of underlying structure).

A realistic example of a non-forecasting time-series task is the seismic classification dataset from the R package **astsa**, where the goal is to distinguish ordinary earthquakes from underground nuclear tests using time-indexed signal data. Similar classification or anomaly-detection problems are entirely acceptable project topics, provided temporal structure and time-aware validation are taken seriously.

Different datasets come with different starting points:

- curated benchmark series (for example widely used climate or economic indicators, or the earthquake vs nuclear test data from **astsa**) may have clearly documented structure and relatively few data-quality problems;

- other sources (for example ad-hoc CSVs, sensor feeds, or scraped data) may require more careful checking and cleaning.

In all cases, the expectation is that you **think systematically** about what your series might contain and what it might miss, rather than assuming that problems are always present.

It is entirely acceptable, given the limited time available, for a project to conclude that **no clear structure beyond simple baselines is found** (whether for forecasting or classification), provided that:

- the data have been examined carefully;
- appropriate baselines and validation schemes have been used;
- limitations and uncertainties are discussed honestly.

The project should tell a clear story:

- what question is being asked (forecasting, classification, anomaly detection, etc.) and why it matters in your domain;
- what data are available, how they are structured in time, and how reliable they appear to be;
- what temporal structure seems to be present (trend, seasonality, dependence, possible regime changes), and how confidently you can say so;
- how your models and validation schemes relate to that structure, and what their limitations are.

1.8.1 8.1 Expectations and first steps

Concrete expectations for your initial work:

- **Data understanding**
 - describe the temporal index and sampling pattern (for example regular vs irregular, frequency, gaps) (see Section 2.1 and Section 3.1);
 - consider whether there might be missing values, anomalies, or regime changes, and check for them where plausible (see Sections 3.2–3.3);
 - distinguish between obviously clean, curated series and more ad-hoc or noisy data sources.
- **Visual EDA**
 - plot levels and, where appropriate, differences or other transformations (see Section 4.1);
 - look for indications of trend, seasonality, changes in variance, and autocorrelation (see Sections 4.2–4.3 and 5.1);
 - note whether rare or extreme events appear in the sample; if they do not, briefly reflect on how that affects your question (forecasting, classification, or anomaly detection) (see Section 3.3).
- **Simple decomposition**
 - where meaningful, attempt to separate trend and seasonality (for example via regression or basic decomposition) (see Sections 5.1–5.2);
 - inspect residuals for remaining structure (autocorrelation, changing variance, unexplained patterns), and be explicit if residuals look essentially like noise (see Section 5.2).
- **Basic baselines**
 - define at least one **persistence** baseline (for example “tomorrow = today”) and any obvious seasonal persistence baselines where relevant for forecasting tasks (see Section

- 6.2);
- for classification tasks, define simple baselines (e.g. majority class, threshold rules, low-dimensional classifiers on basic features) with **time-aware validation** where appropriate (see Section 4.4 and Section 7.1);
- if you use ML methods, start with naive baselines using validation schemes that respect temporal order and avoid leakage (see Sections 4.4 and 7.2–7.3).

1.8.2 8.2 Scientific sources and the use of large language models (LLMs)

You are not forbidden from using large language models (LLMs) or similar tools to:

- generate draft code or boilerplate;
- obtain informal explanations of methods;
- explore alternative ways of structuring your analysis.

However, you should treat these tools as **bullshit generators**: they produce plausible-sounding text and code without any genuine understanding or guarantee of truth. At the time of writing, LLMs are quite capable of:

- inventing confident but incorrect explanations and derivations;
- producing subtly wrong code (especially for time-series indexing, validation, and edge cases);
- fabricating references, data descriptions, and claims that look convincing but are simply made up.

In practical terms:

- any code obtained via LLMs must be **reviewed, tested, and understood** by you;
- any explanation or claim must be **cross-checked** against trusted references (lectures, textbooks, documentation);
- you remain responsible for the correctness of your analysis and conclusions, regardless of where the text or code originated.

Sources and attribution

Independent of whether you use LLMs, you are expected to base your work on **verifiable sources**: for example, published papers, textbooks, official documentation, and the lecture notes. Claims about methods, assumptions, and results should be traceable to such sources, not to LLM output.

If you do use LLMs for drafting:

- treat their output as a starting point that must be checked against real sources, not as an authority;
- cite the actual books, articles, and documentation that you rely on, rather than the LLM itself;
- briefly indicate in your notebook where LLM assistance was used and what you did to verify and correct its suggestions.

Using LLMs as a convenience for drafting is acceptable. Using them as a substitute for reading and understanding real sources is not.

1.8.3 8.3 Companion notebooks and example code

This lecture notebook focuses on concepts and short illustrative examples. For **reusable code** and project-oriented workflows, use the companion notebooks in the same folder:

- PCS956-TS-companion_A Examples related to inspection and exploratory analysis of time-series data.
- PCS956-TS-companion_B Examples of classical time-series models and simple forecasting baselines.
- PCS956-TS-companion_C Examples of machine-learning methods for time series and time-aware validation schemes.

The companion notebooks are intended as **starting points** rather than complete solutions. Their contents may be updated during the module. You are free to adapt any examples they contain to your own dataset and project question.

1.9 9. Looking ahead: forecasting, anomalies, and drift

Later lectures build on the foundations established here: - Lecture TS2 focuses on forecasting baselines, classical and ML models, supervised-learning formulations for time series, temporal validation, and evaluation on levels vs differences. - Lecture TS3 focuses on anomaly detection, concept drift, multivariate and nonlinear dependence, explainability, and high-level research trends.

These topics deepen the ideas of: - data quality and rare events; - structure vs noise and stationarity; - modelling scepticism and scientific thinking.

1.9.1 9.1 Connection to TS2

Lecture TS2 will: - formalise forecasting tasks and baselines; - show how to cast time series problems into supervised-learning form; - emphasise temporal validation and leakage avoidance; - compare evaluation on levels vs differences and other transformations.

By the end of Lecture TS2, students should be able to:

- explain the structure and assumptions of basic ARIMA-type models sufficiently to choose appropriate baselines and transformations;
- recognise when such models are misapplied to clearly non-stationary or drift-dominated series.

The EDA and decomposition tools from Lecture TS1 provide essential context for choosing appropriate baselines and models in TS2.

1.9.2 9.2 Connection to TS3

Lecture TS3 will: - revisit anomalies and regime changes with more advanced detection methods; - examine concept drift and model degradation in production; - explore multivariate and nonlinear dependence, including dimension reduction that respects temporal structure; - discuss explainability and research trends in time series ML.

The data-quality mindset and scepticism about spurious patterns developed here are crucial for interpreting anomaly and drift detection results.

1.10 10. Code templates and practical workflows

This section is provided in the companion notebooks:

- PCS956-TS-companion_A (EDA and inspection),
- PCS956-TS-companion_B (classical models and baselines),
- PCS956-TS-companion_C (ML methods and time-aware validation).

Those notebooks contain reusable code templates for: - applying transformations and decomposition, - fitting models on residuals or transformed series, - reconstructing forecasts and diagnostic plots on the original scale.

1.11 Further Reading and Online Resources

1.11.1 Practical blogs and articles (examples and baselines)

Some of the examples and emphasis on naive / persistence baselines in this lecture were inspired by:

- Vegard Flovik *How (not) to use Machine Learning for time series forecasting: Avoiding the pitfalls* <https://www.kdnuggets.com/2019/05/machine-learning-time-series-forecasting.html>

How (not) to use Machine Learning for time series forecasting: The sequel <https://www.kdnuggets.com/2020/03/machine-learning-time-series-forecasting-sequel.html>

- Jason Brownlee *A Gentle Introduction to the Random Walk for Times Series Forecasting with Python* <https://machinelearningmastery.com/gentle-introduction-random-walk-times-series-forecasting-python/>

1.11.2 Forecasting and statistical time-series methods

- Rob J. Hyndman & George Athanasopoulos *Forecasting: Principles and Practice* (3rd ed) <https://otexts.com/fpp3/>